University of Camerino

SCHOOL OF SCIENCES AND TECHNOLOGY

Master Degree in Computer Science (Class LM-18)
Complex System Design

AI Driven Detection of Abnormal Smart Contract Executions

Group Members
Diego Marinangeli (132676)
Giacomo Calcaterra (134523)
Demetrio Angeloni (134526)
Alessandro Mermali (136584)

Internal Advisor
Prof. Flavio Corradini

External Advisor
Prof. Alessandro Marcelletti
Dr. Filippo Lampa

A.A. 2025/2026

Contents

1 Introduction **7**

2 Project Activities **9**
2.1 Activities Descriptions 9

3 Background **11**
3.1 Blockchain 11
3.2 Smart contract 11
3.3 Decentralized Finance (DeFi) 11
3.4 Vulnerabilities 12
3.5 LLM 12
3.6 Transformer 13
3.7 Encoder-based Transformers (BERT) 14
3.8 Decoder-based Transformers (GPT) 14

4 State of the Art **17**

5 Approach Overview **21**
5.1 Pipeline Description 21
5.2 Interpretation of the architecture 22

6 Dataset **25**
6.1 General Data Structure 25
6.2 Core Datasets 26
6.3 Dataset Expansion 27
6.4 Data Normalization and Schema Mapping 29

7 Architecture **31**
7.1 Tokenizer 31
7.2 Embedding 35
7.3 GPT Architecture 35

8 Implementation **37**
8.1 Technologies and Frameworks used 37
8.2 Repository Structure 37
8.3 Window-Based Architecture and Optimization 38

8.3.1 Context Windows and Block Size 38
8.3.2 Training vs. Inference Strategy 38
8.3.3 Computational Optimization 39
8.4 Key Parameters and Their Influence 39
8.5 Anomaly Scoring 39
8.6 System Usage Instructions 40

9 Training and evaluation **41**
9.1 Setup 41
9.2 Training Results 42
9.3 Anomaly Detection Evaluation 42
9.4 Overfitting Analysis 44

10 Conclusions and future work **45**

List of Figures

2.1 Overview of the project phases and their duration. 9

3.1 The Transformer model architecture. 14

5.1 Pipeline of the proposed framework. 22

6.1 Event log of the Tornado Cash governance attack execution trace. . . . 28

7.1 System architecture 31
7.2 Transaction Tokenizer 32
7.3 Tree Tokenizer 33
7.4 Context Tokenizer 33

9.1 Top 10 detected attack transactions ranked by anomaly score 44

Abstract

The rapid development of blockchain technology and the widespread adoption of smart contracts have revolutionized multiple sectors, introducing benefits such as decentralization, transparency, and immutability.
However, these innovations also introduce significant security risks; once deployed, smart contracts become immutable, meaning that vulnerabilities or malicious behaviors cannot be easily corrected. This makes the detection and analysis of abnormal and fraudulent behavior in blockchain transactions a crucial challenge to prevent irreversible financial consequences and maintain trust in the technology.
To date, the primary techniques used to identify fraudulent contracts rely on static and dynamic code analysis, rule-based systems, and machine learning models based on graphical representations of code features. While many existing approaches perform prior analysis on malicious attacks, they often focus on patterns that lack a close dependence on the internal logic and evolving execution of the code. Consequently, there is a critical need for a detection technique that performs abnormal detection to identify behaviors that deviate from established secure patterns.
By focusing on abnormal detection, we can better understand the history of an attack and perform a more robust analysis compared to traditional prior-analysis techniques For this reason, this project develops a predictive model based on Transformer architectures, specifically utilizing the GPT model.
To evaluate the model's effectiveness, we conducted experiments on a dataset consisting of unlabeled Ethereum blockchain transactions based on established secure transaction history.
The goal is to propose a technique to detect potential abnormalities in the internal logic of transactions which, while not explicitly noticeable in the source code, might still lead to unexpected fraudulent behaviors

1. Introduction

Since the emergence of Bitcoin in 2008, the concepts of Blockchain and Smart Contracts have evolved from simple transactional mechanisms to complex programmable infrastructures capable of supporting entire digital ecosystems.
Blockchain technology, originally conceived as a peer-to-peer electronic cash system, has progressively matured into a decentralized and tamper-resistant computational framework that enables secure, transparent, and verifiable data management across distributed networks. Its decentralized nature eliminates the need for trusted intermediaries, making it suitable for applications far beyond cryptocurrencies, including supply chain management, healthcare, energy markets, and finance.
At the heart of this technological revolution lies the concept of smart contracts, they consist of self-executing agreements encoded within the blockchain that automatically enforce predefined conditions. Platforms such as Ethereum have popularized their use, enabling the rise of Decentralized Applications (DApps) that replicate and extend traditional financial and organizational mechanisms in a trustless and autonomous environment. These contracts encapsulate business logic and execute deterministically when triggered by specific transactions, ensuring full transparency and accountability.
However, the same immutability that makes smart contracts reliable also introduces a critical limitation: once deployed, their code cannot be altered or corrected. This property, while essential to maintaining blockchain integrity, amplifies the consequences of programming flaws or malicious design. Vulnerabilities may result in the loss of funds, market manipulation, or system instability, and have already caused several large-scale incidents in decentralized finance (DeFi) ecosystems*[Boa22]*.
DeFi represents one of the most innovative and fast-growing domains built on blockchain technology. It leverages the composability of smart contracts to recreate financial instruments such as lending, borrowing, trading, and derivatives, without intermediaries or centralized authorities. Yet, this very openness also increases systemic risk: malicious actors can exploit vulnerabilities in smart contracts or protocol logic, leading to fraudulent transactions, liquidity manipulation, and cascading failures across interconnected systems.
For instance, Beanstalk, a permissionless fiat stablecoin protocol, was the victim of a massive $ 181 million hack on April 17, 2022, which leveraged the lack of execution delay to push through a malicious governance proposal*[Sta22]*.
As blockchain adoption accelerates, the need for automated and intelligent vulnerability detection mechanisms has become paramount. Traditional static and dynamic code analyses, while useful, often fail to generalize across the increasingly complex architectures of modern smart contracts.
Analyzing execution data allows the system to understand the complete and effective behavior of a sequence of transactions, capturing the anomaly reactively and immediately.

By adopting AI techniques, it is possible to ingest transaction execution data to the system to perform detection analysis.
Nowadays, there exist several solutions that perform analysis on execution data of transactions, such as BlockGPT*[Gai+23]* and BlockScan*[Yu+25]*. However, these solutions perform a detection analysis after the execution of the transactions. Hence, a malicious transaction may have already infected a smart contract, before it has been detected and resolved.
For this reason, this project explores an AI-driven approach to predict and identify potential vulnerabilities at runtime, leveraging Transformer-based models inspired by the GPT architecture. By learning intricate patterns and long-range dependencies in smart contract logic, such models can detect subtle anomalies that may precede exploitative behaviors.
Due to its architecture, the proposed solution guarantees a faster and accurate detection of the anomaly. Detection analysis can be done reliably, countering the anomaly reactively and immediately.
The detection analysis takes into account several types of execution data, such as transaction hash, sender, receiver, and internal calls of the transaction itself, in order to provide a complete and meticulous detection of the anomaly.
Furthermore, the proposed solution guarantees more security for the user's wallets, protecting them from irreversible conditions that can be caused by blockchain attacks. In this way, users can do transactions without any concern for security problems.

2. Project Activities

In this chapter, the various tasks, that we have performed to develop the proposed solution, have been illustrated. The GANTT diagram in the Figure 2.1 describes the various tasks with relative duration and date time.

Task Name	Start	End	Dur
AI Detection Project	**3/10**	**6/2**	**107**
1. Research on Blockchain	3/10	23/10	20
2. Fraudulent Behavior Analysis	24/10	6/11	13
3. Runtime Vulnerability Det.	7/11	20/11	13
4. Model Development	**21/11**	**8/1**	**41**
4.1 Dataset Collection	21/11	4/12	13
4.2 Transformer Design	5/12	18/12	13
4.3 Model Training	19/12	8/1	15
5. Validation on Known Cases	9/1	5/2	20

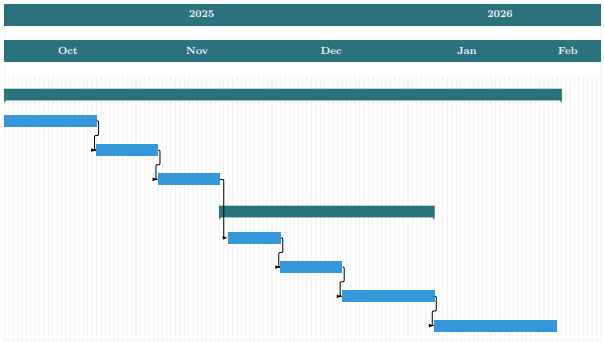

Figure 2.1: Overview of the project phases and their duration.

2.1 Activities Descriptions

Research on Blockchain This phase primarily involves gathering the theoretical background necessary for the study, covering the evolution, structure, and functioning of blockchain and smart contracts, particularly within the context of Decentralized Finance. The goal is to develop a robust knowledge base and gather the core information presented in Section 3.1.

Fraudulent Behavior Analysis This activity focuses on analyzing specific security risks and malicious patterns in the blockchain ecosystem. It involves studying common vulnerabilities that emerge only at runtime, like Wash Trading, Pump-and-Dump Schemes, Rug Pulls, Honeypots and Reentrancy attacks. This is done to better understand the nature and the type of abnormal activities the model needs to detect. The result of this research are presented in the Section 3.4.

Runtime Vulnerability Detection This phase addresses the limitations of traditional analysis by exploring runtime detection methods. The goal is to define an AI-driven approach capable of identifying anomalies in internal contract logic and execution flows as they occur. This is done by analyzing and comparing similar, previously done work and highlighting their strengths and limitations. This work is presented in the Chapter 4.

Dataset Collection This phases focuses on the aggregation and curation of data essential for the model's training process. The objective is to construct a unified dataset of smart contract execution traces capable of representing both simple and complex behavioral patterns across different domains. The specific sources, along with the collection and preprocessing methodologies employed, are detailed in Chapter 6.

Transformer Design Designing the architecture of the custom GPT-based model. This includes defining specific tokenizers and embedding layers to convert transaction tokens into vector spaces for the neural network. It also includes the definition of a likelihood metric to evaluate the abnormality of a given transaction. The architecture is discussed in detail in Chapter 7.

Model Training The process of training the GPT model using the processed vector spaces. The model is trained to recognize long-range dependencies and patterns in transaction history to predict legitimate versus anomalous next tokens. The training process and the parameters used are further explained in Chapter 10.

Validation on Known Cases The final phase involves stress-testing the model against labeled or well-known datasets, such as historic case logs. This step verifies the system's reliability in detecting complex, real-world attacks and anomalies that were identified during the dataset expansion phase. The results are presented in Chapter 10.

3. Background

This chapter provides an overview of the key concepts and prior work that form the foundation of this study. It summarizes the essential theoretical background, highlights relevant developments in the literature, and outlines the context in which the research is situated. The goal is to offer readers a clear and concise understanding of the groundwork necessary to follow the remainder of the paper prior studies necessary to contextualize the work.

3.1 Blockchain

The concept of Blockchain was introduced by Satoshi Nakamoto in "Bitcoin: A Peer-to-Peer Electronic Cash System" in 2008 *[Nak08]*. A blockchain can be viewed as a decentralized database *[WLZ21]* in which blocks are linked through cryptographic mechanisms, typically including the hash of the previous block to ensure integrity and immutability. Each block contains a set of transactions and is securely linked to the one before it while being replicated across a distributed network of nodes. It also eliminates the need for third-party intermediaries and enables systems in which transparency, security, and accountability are built into the design *[Rac25]*.

3.2 Smart contract

In 1997, "smart contracts" were explained by Nick Szabo *[Sza97]*. Smart contracts are included in the blockchain and are pieces of code that can be automatically executed without intermediaries and consist of value, address, functions, and state variables *[Mad16]*.Once deployed, a smart contract becomes immutable and its execution is enforced by the underlying blockchain. However, this immutability does not exclude the possibility of destruction if the contract contains specific logic for it. When such a mechanism is triggered, the contract is permanently removed, can no longer be called, and any remaining funds are sent to a designated address. Smart contracts are typically written in high-level, platform-specific languages that are compiled into low-level bytecode. This bytecode is then published on the blockchain through a deployment transaction *[TS+19]*

3.3 Decentralized Finance (DeFi)

Decentralized Finance (DeFi) builds upon blockchain technology to create an open, permissionless financial ecosystem that operates without traditional intermediaries such as banks or lending institutions. It enables users to access financial services directly

through smart contracts, which automate operations like lending, borrowing, trading, and asset management. By leveraging transparency and composability, DeFi promotes innovation and accessibility, allowing anyone with an internet connection to participate in global financial activities.

3.4 Vulnerabilities

From the literature reviewed during the preliminary study phase of this work, several vulnerabilities and manipulation schemes have emerged as particularly recurrent. Among the most relevant are the following.

Wash Trading. consists of repeatedly buying and selling the same asset to oneself, often through multiple controlled addresses, to artificially inflate trading volume or create the illusion of market demand. Under the U.S. Commodity Exchange Act, it is formally defined as "entering into, or purporting to enter into, transactions to give the appearance that purchases and sales have been made, without incurring market risk or changing the trader's market position." This practice distorts price discovery, misleads investors, and can be used to boost the perceived popularity of a token or NFT project.

Pump-and-Dump Schemes. Pump-and-dump schemes involve artificially inflating the price of a token through coordinated buying, misleading marketing, or exaggerated claims, followed by a sudden sell-off by the manipulators once the price peaks. This causes the token's value to crash, leaving unsuspecting investors with significant losses. These schemes are facilitated by low-liquidity markets and the ease of creating new tokens with minimal oversight.

Rug Pulls. A rug pull occurs when the developers of a project deliberately withdraw liquidity or abandon the project after attracting investors, causing the token's value to collapse. This type of scam exploits the asymmetry of information between project creators and users, especially in decentralized exchanges where liquidity pools can be drained instantly. Rug pulls are particularly frequent in newly launched or unaudited tokens, where trust relies solely on the developers' claims.

Honeypot. A honeypot is a malicious smart contract designed to appear exploitable, thereby luring attackers into interacting with it. While the contract seems to allow unauthorized token withdrawals or profitable arbitrage, hidden logic prevents the victim from completing the transaction, causing their funds to be trapped. Meanwhile, the contract creator retains full control and can extract the funds at any time. Honeypots rely on deceptive code patterns and exploit users' assumptions about vulnerabilities.

Reentrancy. Reentrancy is a classic smart contract vulnerability in which an external contract repeatedly calls back into a function before previous operations have been completed. If state updates occur after external calls, an attacker can repeatedly withdraw funds or trigger unintended behavior during these recursive calls. The most notable example is the 2016 DAO attack, where reentrancy allowed an attacker to drain millions of dollars in Ether. Mitigation typically involves reordering operations or using reentrancy guards.

3.5 LLM

Research has shown that significantly increasing model size or dataset scale improves the ability to handle complex language tasks, including translation, summarization,

information retrieval, conversational interaction, writing, and code completion. The term Large Language Model (LLM) refers to large-scale Pre-Trained Language Models (PLMs) based on advanced deep learning techniques. LLMs stand out for their ability to generate high-quality, human-like outputs thanks to scalability and the emergence of capabilities that typically do not appear in smaller models *[DR+25]*.

In practice, many modern LLMs are built on Transformer architectures and, during pre-training, learn syntactic and semantic regularities directly from massive text collections. Once pre-trained, they can be adapted to specific downstream tasks through supervised fine-tuning or guided via prompting, leveraging the general knowledge acquired during training. This combination of large-scale pre-training and targeted adaptation makes LLMs flexible tools that can generalize across different settings with lower development costs than training task-specific models from scratch.

3.6 Transformer

Transformers *[Vas+17]* are deep learning architectures that replace the need for Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), and have rapidly become one of the most influential milestones in the development of modern Natural Language Processing (NLP). Transformer architecture also introduces full parallelization during training, significantly improving computational efficiency and enabling models to learn long-range dependencies more effectively than traditional sequential approaches.

At the core of the architecture lies the self-attention mechanism, which enables the model to evaluate the relevance of each token with respect to all others in the same sequence. As illustrated in Figure 3.1, the model is composed of stacked encoder and decoder blocks, each incorporating multi-head self-attention and feed-forward layers. By learning these global dependencies directly, Transformers can capture long-range relationships without relying on recurrent computation, making them particularly effective when important cues are distributed across distant parts of the input.

The Transformer paradigm has given rise to several model families, including encoder-based approaches (e.g., BERT) and decoder-based approaches (e.g., GPT), which differ mainly in how they are trained and how they represent context. In this work, we ultimately rely on a GPT-style, autoregressive formulation to model ordered interaction traces (such as sequences of transactions). This choice is particularly well-suited to the blockchain domain: transactions are inherently sequential and timestamped, meaning that temporal directionality is a structural property of the data rather than a modeling assumption. Furthermore, the autoregressive nature of the model enables incremental, *runtime* detection of suspicious activity, a critical advantage in DeFi settings where exploits can propagate irreversible consequences within a single block. The detailed architectural motivation is discussed in sections 3.7 and 3.8.

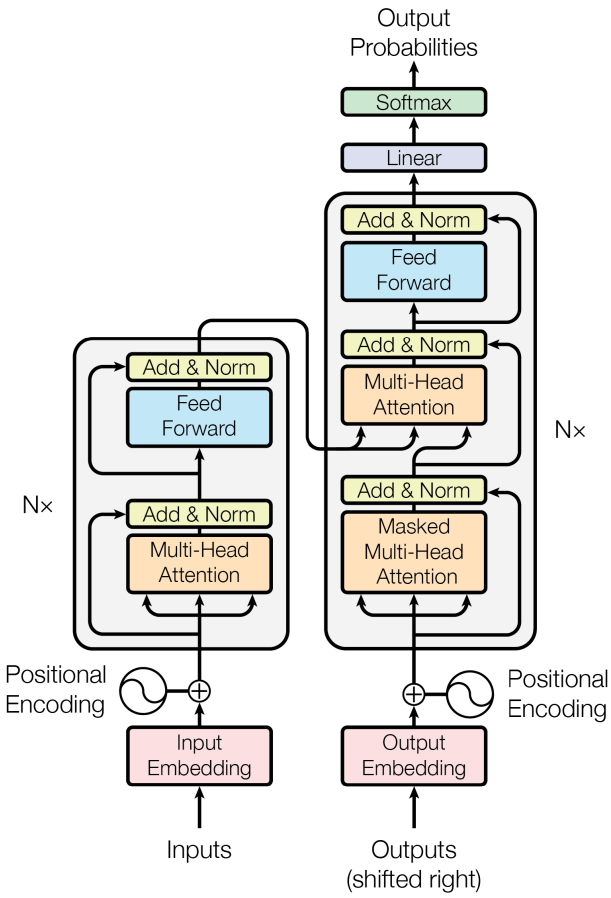

Figure 3.1: The Transformer model architecture *[Vas+17]*.

3.7 Encoder-based Transformers (BERT)

Encoder-based models operate over an input sequence $\mathbf{x} = (x_1, \ldots, x_n)$. For a given token x_i, we define its *left context* as the tokens preceding it, i.e., $(x_1, \ldots, x_{i-1})$, and its *right context* as the tokens following it, i.e., $(x_{i+1}, \ldots, x_n)$. BERT *[Dev+19]* is a representative model of this family: it build contextual representations by attending to both left and right context simultaneously. They are commonly pre-trained with a masked language modeling objective, where a subset of tokens is masked and the model learns to reconstruct them using the surrounding context.

Because this bidirectional representation is well suited to understanding and classifying complete sequences, BERT-style models are often adopted in tasks where the input is available in full and the goal is to infer properties of the entire text (e.g., classification or extraction). In the context of blockchain security, this paradigm naturally aligns with settings where one analyzes a complete artifact (for instance, a contract or a code fragment) without explicitly emphasizing temporal progression.

3.8 Decoder-based Transformers (GPT)

Decoder-based models, such as GPT, are trained with an autoregressive objective: given a sequence prefix, the model predicts the next token. This implies a causal (left-to-right) attention pattern, which makes the learned representation inherently order-aware.

In this project, we adopt a GPT-based architecture, since our data can be framed as a sequence of events (e.g., transactions or interactions) in which temporal order carries meaningful information. This choice supports modeling the "flow" of behavior and helps highlight deviations from typical event patterns. Moreover, self-attention allows the model to dynamically weigh which parts of the history are most relevant, enabling the integration of heterogeneous signals (temporal, structural, and behavioral) into a single representation.

4. State of the Art

This chapter discusses the State of the Art (SOTA) on the specific task of anomaly detection and fraud analysis in the DeFi ecosystem.

Reference	Paradigm	Strengths	Key Gap vs. Our Work
[Con+23]	Statistical	Interpretable market irregularities.	Retrospective; lacks real-time runtime monitoring.
[Suz+]	Static	Off-chain/On-chain integration.	Rigid heuristics; easily evaded by novel tactics.
[MAD22]	Classical ML	High accuracy for known scams.	Dependent on labeled history; poor for novel attacks.
[Man+25]	Process Mining	Discovers motifs without labels.	Computationally heavy; lacks real-time scalability.
[Zhu+20]	GNN	Contract-level vulnerability.	Fails to model temporal sequences of transactions.
Our Approach	**GPT-based**	**Models long-range temporal and structural patterns.**	**Runtime detection via enriched internal call decoding.**

Table 4.1: State-of-the-Art Analysis: Comparison of Blockchain Detection Methodologies

Research on fraud and vulnerability detection in blockchain varies across a wide methodological spectrum, reflecting the heterogeneity and rapid evolution of illicit behaviors across decentralized platforms. Although these paradigms differ in scope and technical depth, together they outline the trajectory of how the field has progressively moved from retrospective heuristic-driven techniques toward more adaptive, real-time, and data-driven detection frameworks.

Table 4.1 presents a comparative analysis of existing blockchain detection methodologies. It synthesizes the core paradigms, strength and key gaps of the reviewed literature, highlighting how our proposed GPT-based approach addresses the need for real-time runtime monitoring.

Early work by Cong et al. *[Con+23]* exemplifies the statistical tradition in blockchain fraud analysis. Their study formalizes wash trading detection in centralized exchanges by tracking behavioral and market-structure irregularities that deviate from the expected patterns of well-functioning markets. By explicitly modelling distributions of order flows, liquidity provision and transaction timing, they are able to flag anomalous activity suggestive of artificial volume inflation. This approach is particularly strong in its interpretability: the flagged events correspond to economically meaningful devi-

ations that can be directly understood by regulators and market analysts. However, since it relies on aggregate historical patterns and assumes relatively stable market microstructure, its applicability is largely retrospective and poorly suited for real-time monitoring in more volatile decentralized settings.
Cong et al. and Suzuki et al. *[Suz+]* both operate within the broader category of static analysis, though with different emphases. While Cong et al. focus primarily on exchange-level irregularities, Suzuki et al. *[Suz+]* introduce DeFiIntel, a large dataset that systematically links on-chain token transactions with off-chain artefacts (websites, social media). Their methodology incorporates transaction graph features, code-level signals and external indicators like social media metrics. However, this static rule-driven paradigm remains constrained by its need for manually defined heuristics. These heuristics are often tightly coupled to known attack patterns, which reduces adaptability when adversaries deliberately evolve strategies to evade such detection.
The shift toward machine learning is exemplified by the "Do Not Rug on Me" study *[MAD22]*, which defines a taxonomy of rug pulls and other malicious moves on Uniswap and trains ML models on Uniswap transaction data. By designing features that capture token propagation and contract heuristic, they are able to flag many scam tokens before the rug event, achieving very high accuracy. However, the approach still relies on rich historical transactions traces and careful labelling, which limits its direct applicability to novel token environments. Similarly, Onu et al. *[Onu+23]* investigate the detection of Ponzi schemes through classical ML models, exploring dimensionality reduction techniques to mitigate overfitting and address severe class imbalance. While these machine-learning approaches clearly outperform simple heuristic-based systems in flexibility and predictive power, they remain fundamentally dependent on the availability of labelled historical data. This dependence makes them predominantly retrospective: fraudulent behaviors are flagged after they manifest, rather than during their execution.
Manzor et al. *[Man+25]* introduce a different perspective by applying process mining to blockchain event logs. Their method reconstructs behavioral traces, uncovering recurrent interaction patterns that may indicate coordinated manipulation. This is particularly effective for identifying clusters of wallets engaging in cyclical behaviors or suspiciously repetitive token interactions. The strength of this approach lies in its ability to reveal macro-level behavioral motifs without requiring labelled data. However, process mining is computationally expensive and inherently retrospective, making it difficult to deploy for continuous, real-time fraud detection at blockchain scale.
A more structurally grounded approach appears in the work of Zhuang et al. *[Zhu+20]*, who introduce a *contract graph* representation that captures both syntactic and semantic relationships within Solidity smart contracts. By training a Graph Neural Network (GNN) over this structure, they achieve promising performance in vulnerability classification. This direction exploits the relational nature of smart-contract code and can detect vulnerabilities that may lead to exploits even before contracts are deployed. Yet, as powerful as GNNs are for code-level vulnerability detection, they have a limited capacity to model temporal sequences of transactions, which restricts their usefulness for detecting runtime fraudulent behaviors like pump-and-dump schemes, wash trading or dynamic liquidity manipulation.
More recent work points toward Transformer-based models as the most promising paradigm for real-time fraud detection in blockchain environments. Transformers excel at modelling long-patterns that unfold over extended time horizons. Their ability to integrate temporal, structural and behavioral signals into a unified representation

allows them to detect subtle anomalies that elude both rule-based and classical ML systems. Despite these advantages, Transformer models require meticulously constructed datasets, significant computational resources and careful handling of sequential tokenization for blockchain-specific data formats.

Taken together, these studies illustrate a clear trend in the field: techniques have transitioned from interpretable but rigid statistical and heuristic methods toward more adaptive and expressing machine learning and deep learning frameworks. Static and statistical approaches offer clarity and reliability but cannot keep pace with evolving fraud tactics. Classical ML improves generalization but relies heavily on labelled datasets. Process mining and GNNs expand the analytical space to behavioral and structural dimensions, yet struggle with scalability and temporal reasoning. Transformer-based architectures appear to offer the best foundation for real-time, high-resolution fraud detection, provided that the necessary data pipeline and computational infrastructure are in place.

5. Approach Overview

The present work adopts a runtime-oriented analysis framework for the detection of abnormal smart contract executions through the systematic analysis of structured execution traces. This methodological choice is aligned with the objectives of the project and with the adopted pipeline presented during the design phase, where blockchain data is processed through a sequence of interconnected modules that distinguish between training time and inference time while sharing a common representation space.

In contrast to traditional approaches that focus on static code inspection or isolated transaction analysis, our methodology considers each smart contract execution as a complex behavioral sequence. Every transaction on the blockchain generates a rich set of execution signals derived from the Ethereum Virtual Machine (EVM), including input parameters, internal call hierarchies, gas consumption, and emitted events. These elements collectively describe not only what a transaction does, but also how it evolves during execution.

By modeling execution traces as structured behavioral artifacts rather than flat records, the system is able to capture both structural dependencies and contextual semantics. This perspective is particularly relevant in decentralized environments, where malicious behaviors and exploits often emerge from multi-step execution flows rather than from single anomalous values. Consequently, the proposed approach emphasizes sequential pattern learning and contextual representation of smart contract activity at runtime.

5.1 Pipeline Description

Figure 5.1 illustrates the overall architecture of the proposed framework. The diagram reflects the conceptual workflow adopted in the project and maintains a clear distinction between the training phase and the inference phase, while ensuring that both rely on the same data representation module.

More specifically, historical data is first collected and processed during training time in order to construct a coherent and normalized dataset of smart contract execution traces. In parallel, runtime data is continuously fed into the system during inference, following the same preprocessing and representation steps. This architectural symmetry is a deliberate design choice, as it guarantees consistency between the data used to learn behavioral patterns and the data observed during real-time detection.

The pipeline can therefore be interpreted as a unified behavioral analysis loop in which both past and live execution traces are mapped into a shared semantic space before being analyzed by the detection model.

Historical data is used during training to build the dataset through the data collection module, where heterogeneous execution traces are extracted, cleaned, and organized

into a unified structure. At the same time, runtime data is directly forwarded to the data representation block, ensuring that real-time transactions undergo the same transformation process adopted during training.

The core of the pipeline is represented by the data representation module. This component is responsible for transforming raw EVM traces into structured and machine-readable sequences. In our methodology, this process includes normalization, formatting, tokenization, and embedding generation. Raw traces, which are originally noisy and heterogeneous, are converted into clean sequences that preserve the semantic meaning of transaction inputs, internal calls, and emitted events.

Following this transformation, embeddings are generated to encode the behavioral characteristics of each execution trace into dense numerical vectors. These embeddings act as a compact representation of transactional semantics, structural call-tree information, and contextual features derived from the execution environment.

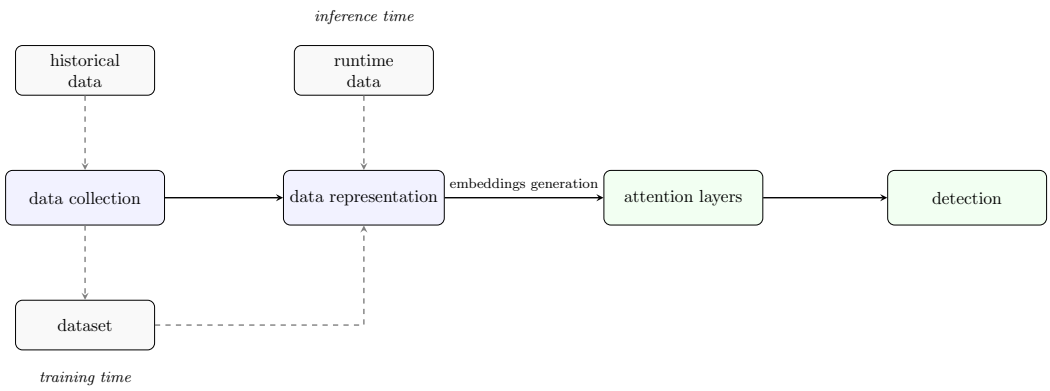

Figure 5.1: Pipeline of the proposed framework.

5.2 Interpretation of the architecture

As shown in the figure 5.1, the architecture is explicitly divided into two complementary operational phases: training and inference. This separation is not merely conceptual, but functionally embedded in the design of the pipeline.

During the training phase, historical blockchain data originating from heterogeneous smart contracts is collected through the data collection module. These data sources may include contracts from different domains such as DeFi, governance protocols, automated market makers, and known exploit scenarios. The collected traces are then organized into a structured dataset that preserves key execution attributes, including metadata, internal transactions, and event logs. This curated dataset serves as the foundation for learning normal and abnormal behavioral patterns.

During inference time, runtime data is processed using the same representation pipeline employed during training. Importantly, the system does not introduce alternative preprocessing logic for live data. Instead, runtime traces are normalized, tokenized, and embedded within the same representation space learned from historical data. This design ensures representational coherence and minimizes distribution shifts between offline learning and real-time analysis.

The attention layers constitute the analytical core of the framework. Based on Transformer architectures, these layers perform sequential pattern analysis over the embedded execution traces. Through the self-attention mechanism, the model is able

to capture long-range dependencies, hierarchical execution relationships, and subtle correlations between different transaction components. This capability is particularly crucial in blockchain environments, where anomalous behaviors often involve complex multi-call interactions and delayed side effects.

Finally, the output of the attention mechanism is fed into the detection module, which computes an anomaly score associated with each runtime execution trace. This score reflects the degree of deviation from learned behavioral patterns and enables the identification of abnormal smart contract executions, including hidden exploits, atypical control flows, and suspicious transactional dynamics that may not be detectable through static analysis alone.

Overall, the proposed pipeline emphasizes behavioral consistency, structural awareness, and runtime adaptability. The architecture is intentionally designed to treat execution traces as rich behavioral sequences and to analyze them within a unified semantic framework.

Such a design naturally highlights the central importance of the dataset construction process. Since the effectiveness of the representation and detection stages strictly depends on the quality, consistency, and completeness of the collected execution traces, a rigorous dataset design and normalization strategy becomes a fundamental prerequisite. For this reason, the chapter 6 provides a detailed description of the dataset, its unified schema, the data collection methodology, and the normalization procedures adopted to support the representation and anomaly detection framework.

6. Dataset

The construction of the dataset is guided by a central objective: enabling the detection of anomalous smart contract behavior through the analysis of execution traces. Rather than focusing only on isolated transactions, our approach models the dynamic behavior of contracts as structured sequences of actions and side-effects. From a conceptual perspective, every blockchain transaction can be interpreted as a small program execution. It has an initial intent (what the caller requests), an execution path (how the contract processes that request), and observable consequences (the resulting state changes and emitted signals). By collecting and organizing these elements across multiple domains, we aim to build a behavioral reference model that captures both legitimate operational patterns and exploit scenarios.

The dataset is therefore designed not simply as a collection of raw logs, but as a curated behavioral archive. Each trace contributes to a growing representation of how real-world smart contracts operate under normal and adversarial conditions. This holistic perspective is essential for training a system capable of recognizing subtle deviations that may indicate hidden vulnerabilities or ongoing attacks. The following sections describe how data is collected, structured, normalized, and expanded to support this behavioral modeling framework.

This chapter details the data collection, preparation, and transformation processes. We aggregated smart contract execution traces from diverse domains—including Governance, DeFi, AMMs, and Exploits—to build a unified repository for anomaly detection. Each trace provides a comprehensive record of contract activity, capturing the operational logic and internal state transitions.

6.1 General Data Structure

A joint observation of the datasets reveals a recurring schema. Despite the heterogeneity of the source contexts, the files share a uniform internal structure that enables cross-domain reading and coherent comparison. Each transaction is described through a set of constant fields: the transaction hash, the address of the involved contract, block height, timestamp, gas consumption, and caller identity.

At a conceptual level, each execution trace can be seen as a layered record of what happens when a smart contract runs. A transaction does not consist of a single action: it often triggers a chain of internal operations, where contracts call other contracts and emit observable signals. These internal steps form an execution tree that reflects the real logic of the system.

To capture this complexity, our dataset preserves three complementary perspectives: the external transaction request, the sequence of internal operations triggered during execution, and the observable outcomes produced as events. Together, these elements

describe both the intention of the caller and the actual behavior of the contract.

From a technical standpoint, the system supports both standard JSON arrays and NDJSON (Newline Delimited JSON) streams. This dual-format ingestion strategy allows efficient processing of very large collections of traces without exhausting memory resources. While the storage format is optimized for scalability, the primary focus remains on preserving the semantic structure of execution traces so that behavioral patterns can be analyzed consistently across domains.

6.2 Core Datasets

The training database is built upon four primary pillars, each representing a specific logical structure and risk profile:

ENS Governance Dataset

Focusing on an ENS governance contract, this dataset is characterized by extensive sequences of voting transactions (`castVote`). It is used to model hierarchical administrative calls and voting power distribution, allowing the model to distinguish complex but legitimate administrative logic from unauthorized exploits.

MakerDAO / DAI Dataset

This dataset contains a significant volume of calls dominated by the `hope` function, used to regulate permissions. The analysis reveals a linear pattern of authorization state changes without complex control flows, providing a baseline for standard permission management toward third-party contracts.

Beanstalk Exploit Analysis

This dataset serves as the "ground truth" for anomalous behavior. It features extremely long internal call sequences, massive gas usage, and atypical control flows indicative of flash loan attacks and governance hijacks. It is essential for verifying the system's ability to identify hidden correlations within complex execution trees.

UniswapV2 Pool Dataset

Centered on a USDC–WETH pair, this dataset focuses on `approve` and `transfer` operations. It models Automated Market Maker (AMM) invariants and the risks associated with "infinite allowances" (`uint256(-1)`), which are often precursors to liquidity manipulation.

6.3 Dataset Expansion

To evaluate the system's performance across different transaction volumes and complexities, we introduced additional execution logs for stress-testing and fine-tuning. These supplementary traces were designed to simulate heterogeneous workload conditions and varying levels of structural complexity. The following table compares the features of our curated dataset against standard public blockchain data sources, highlighting the added value of internal call decoding.

Feature / Data Component	Public Dataset	Our Dataset from *[Cor+25]*
Public transactions	✓	✓
Internal transactions	X	✓
Decoded event parameters	X	✓
Unified schema across sources	X	✓
Multi-domain coverage	X	✓

Table 6.1: Comparative analysis between standard public data and our enriched dataset.

The original core datasets were extended with additional execution logs designed to simulate different operational conditions and workload intensities. This expansion phase aims to test the model not only on domain diversity but also on variations in trace size, structural complexity, and transaction density. In particular, three additional data sources were introduced: a repository of malicious traces (`AttackLogs`) and two large-scale execution logs, namely `LogNuovoGrande.json` and `LogNuovoPiccolo.json`. These datasets are not merely quantitative additions, but serve distinct methodological purposes within the experimental pipeline.

Additional Execution Logs

To further assess the system under different data scale conditions, two supplementary log files were incorporated into the dataset: `LogNuovoGrande.json` and `LogNuovoPiccolo.json`. These files represent newly collected execution traces characterized by different volume and granularity levels.

The file `LogNuovoGrande.json` contains a high-volume collection of transaction traces, featuring longer execution sequences, deeper internal call hierarchies, and a higher density of emitted events. Its primary role is to stress-test the ingestion pipeline and evaluate the model's ability to process complex and large-scale behavioral patterns without performance degradation.

Conversely, `LogNuovoPiccolo.json` consists of a more compact set of traces with shorter execution paths and reduced structural complexity. This dataset is used as a controlled baseline to verify the stability of feature extraction, normalization procedures, and anomaly scoring under lighter workloads. Together, these two logs enable a comparative analysis across different data regimes. While `LogNuovoGrande.json` tests scalability and robustness in high-complexity environments,

`LogNuovoPiccolo.json` supports fine-tuning and validation of the preprocessing pipeline on smaller, more interpretable traces. By integrating these additional logs into the dataset architecture, the system can be evaluated across heterogeneous conditions, ranging from standard operational traces to large-scale executions and adversarial scenarios, thus improving the reliability and generalization capability of the anomaly detection framework.

AttackLogs

A dedicated directory named `AttackLogs` was created to store transaction traces associated with malicious or anomalous smart contract behavior. These logs include exploit-like execution patterns, irregular internal call structures, and atypical event emissions that deviate from standard operational flows. From a methodological standpoint, the AttackLogs dataset is strictly excluded from the training phase and is used exclusively for validation. This separation is intentionally enforced to prevent data leakage and to ensure that the model does not learn exploit signatures directly during training. Instead, the system is trained on predominantly legitimate behavioral traces and later evaluated on previously unseen malicious executions. This design allows the validation process to more accurately reflect real-world deployment scenarios, where anomalous behaviors must be detected without prior exposure during the learning phase.

Finally, the dataset is partitioned at the repository level to ensure methodological rigor: core datasets and additional execution logs (LogNuovoGrande.json and LogNuovoPiccolo.json) are used during training, while the AttackLogs directory is reserved exclusively for validation and never exposed to the learning phase.

As a representative example, Figure 6.1 shows the event log extracted from one of the attack traces stored in the `AttackLogs` directory. Specifically, it captures the critical execution phase of the Tornado Cash governance exploit, where a malicious proposal is atomically executed and its consequences become visible through the emitted events.


```
"events": [
  {
    "eventName": "ProposalExecuted",
    "eventValues": {···
    },
    "eventFrom": "0x5efda50f22d34f262c29268506c5fa42cb56a1ce"
  },
  {
    "eventName": "RelayerBalanceNullified",
    "eventValues": {···
    },
    "eventFrom": "0x58e8dcc13be9780fc42e8723d8ead4cf46943df2",
    "eventSignature": "84"
  },
  {
    "eventName": "RelayerBalanceNullified",
    "eventValues": {···
    },
    "eventFrom": "0x58e8dcc13be9780fc42e8723d8ead4cf46943df2",
    "eventSignature": "85"
  },
  {
    "eventName": "RelayerBalanceNullified",
    "eventValues": {···
    },
    "eventFrom": "0x58e8dcc13be9780fc42e8723d8ead4cf46943df2",
    "eventSignature": "86"
  },
  {
    "eventName": "RelayerBalanceNullified",
    "eventValues": {···
    },
    "eventFrom": "0x58e8dcc13be9780fc42e8723d8ead4cf46943df2",
    "eventSignature": "87"
  },
  {
    "eventName": "Transfer",
    "eventValues": {
      "0": "0x2FC93484614a34f26F7970CBB94615bA109BB4bf",
      "1": "0x5efda50f22d34F262c29268506C5Fa42cB56A1Ce",
      "2": 1.6462352310051534e+21,
      "__length__": 3,
      "from": "0x2FC93484614a34f26F7970CBB94615bA109BB4bf",
      "to": "0x5efda50f22d34F262c29268506C5Fa42cB56A1Ce",
      "value": 1.6462352310051534e+21
    },
    "eventFrom": "0x77777feddddffc19ff86db637967013e6c6a116c",
    "eventSignature": "88"
```


Figure 6.1: Event log of the Tornado Cash governance attack execution trace.

The event sequence exhibits three key anomalous signals: a cascade of **RelayerBal-**

anceNullified events targeting multiple legitimate relayer addresses, a final **Transfer** of approximately **1,646 TORN** tokens to the attacker's address (roughly $1.6 million), and the fact that all operations are atomically triggered by a single **ProposalExecuted** event. Such a combination is structurally absent in any legitimate governance transaction, as will be shown in Chapter 9, this trace was identified as the highest-scoring anomaly among all evaluated attack transactions (see Figure 9.1).

6.4 Data Normalization and Schema Mapping

The transition from raw blockchain traces to a format suitable for machine learning requires a rigorous normalization process. This phase ensures that heterogeneous data from different protocols speaks a "common language", effectively mapping JSON fields into a structured sequence for the analysis layers. We utilize specific delimiters that act as "grammatical markers", allowing the system to distinguish between the *intent* of the caller and the *consequences* of the execution.

To preserve the semantic context of each transaction, raw data is encapsulated using the following markers:

- **Input Parameters (`[INsSTART]` / `[INsEND]`)**: these tags isolate the arguments passed to the function, enabling the model to correlate initial parameters with the resulting execution flow.
- **Execution Events (`[EVsSTART]` / `[EVsEND]`)**: these tags encapsulate emitted event values. As events represent the side-effects of a transaction (e.g., token transfers), they are crucial for detecting discrepancies between claimed and actual contract behavior.

The following table details the functional interpretation of each recurring field. This mapping is fundamental as it transforms static metadata into dynamic features used to calculate the final anomaly score:

Field / Structure	Description and Role in Detection
`hash`	Unique identifier used to maintain traceability and prevent duplicates during training.
`gasUsed`	A numerical proxy for execution complexity; spikes often indicate recursive loops or complex exploit logic.
`input` / `internalTxs`	Represents the hierarchical path of the transaction; the primary input for structural and call-tree analysis.
`type`	Classifies the call nature (e.g., `delegatecall`), helping the model recognize patterns of logic hijacking.
`events`	The structured output of the execution; deviations from historical event patterns provide the strongest signal for anomaly detection.

Table 6.2: Mapping of dataset elements to their respective roles in the detection model.

By combining domain-specific data with these structural markers, we obtain a high-fidelity representation of Ethereum Virtual Machine (EVM) state changes. This ap-

proach allows the system to ignore syntactical noise and focus on the behavioral correlations that distinguish legitimate transactions from sophisticated exploits.

7. Architecture

In this chapter has been defined the architecture of the final proposed solution. Initially, a tokenization technique was applied for each transaction, provided by the dataset described above, to split it into elements called tokens. Then, an embedding layer is used to convert the tokens to the respective vector spaces, for a better understanding of the similarity and relationships between tokens. Finally, the vector spaces were passed as input for the training of the custom transformer based on GPT.
In the Figure 7.1, it is possible to see a general overview of the system architecture.

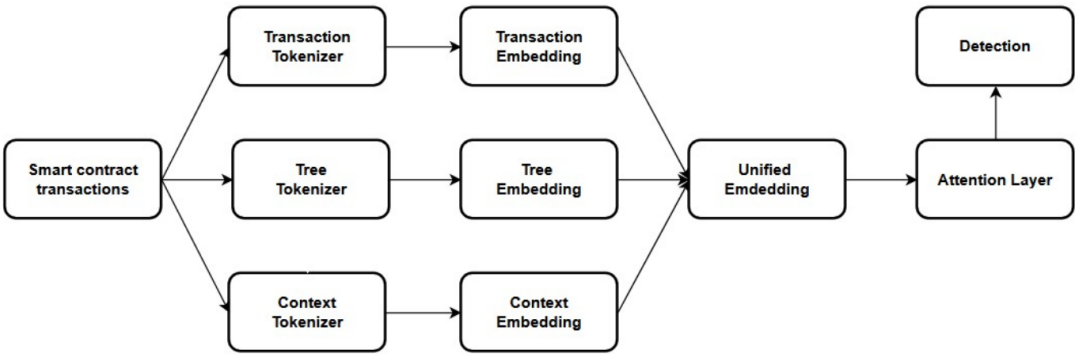

Figure 7.1: System architecture

Below, there is a thorough description about the techniques and transformer cited above.

7.1 Tokenizer

In the context of NLP, a tokenizer is a process that breaks down text into smaller units called "tokens", which can be words, sub-words, or characters, to prepare the text for analysis and processing by models.
In the proposed solution, a given transaction has been split in n elements, called tokens. For each transaction, there have been defined three types of tokenizer with the same length n:

- **Transaction tokenizer**: This tokenizer contains every tokens of the transaction in a string format. There have been inserted extra tokens to define the start and the end of the transaction and to separate information of different fields of the transaction, such as: `inputs`, `internalTxs`, `calls`, `events`. To reduce the computational process, some integer values have been converted into strings, such as `[INT_0]`,`[INT_1]`, `[INT_LT10]`, ..., `[INT_LT1B]`, in order to preserve the

length of the integer value, without explicitly computing it.
Instead, for medium integers we kept exact value, to avoid over-bucketizing and losing anomaly reports based on specific values.
Furthermore, the string tokens, referred to the attributes `transactionHash`, `sender`, and `contractAdrress`, have been converted, respectively, as follows: `[TXHASH]`, `[ADDR]`.

```
['[START]',
 'functionName',
 'propose',
 'transactionHash',
 '[TXHASH]',
 'contractAddress',
 '[ADDR]',
 'sender',
 '[ADDR]',
 'gasUsed',
 '[INT_LT1M]',
 'blockNumber',
 '[INT_LT1B]',
 'timestamp',
 '2023-05-13T07:22:59.000Z',
 'value',
 '[INT_0]',
 '[INsSTART]',
 'type',
 'address',
 'type',
 'string',
 '[INsEND]',
 'storageState',
 '[LIST]',
```

Figure 7.2: Transaction Tokenizer

- **Tree tokenizer**: This tokenizer takes into account only the token regards the `call_id` field of the transaction, to define the hierarchy of the internal transactions respect to the "root" transaction.

```
'0',
'0',
'0',
'0',
'0',
'0',
'0',
'0',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
'0_1',
```

Figure 7.3: Tree Tokenizer

- **Context tokenizer**: This tokenizer takes into account only the tokens of the values of `sender` and `contractAddress` fields, and covert them in TO (the value of `contractAddress`) and FROM (the value of `sender`).

```
'NONE',
'TO',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'NONE',
'FROM',
'NONE',
```

Figure 7.4: Context Tokenizer

Before applying the embedding operation, each tokenizer has been converted into a dictionary key-value, where the key specifies the position of the token in the dictionary and the value specifies the value of the token. In this way has been defined the vocabularies of the tokenizers, that can be seen below.

```
"out_vocab": {
  "[PAD]": 0,
  "[UNK]": 1,
  "[START]": 2,
  "functionName": 3,
  "lock": 4,
  "transactionHash": 5,
  "[TXHASH]": 6,
  "contractAddress": 7,
  "[ADDR]": 8,
  "sender": 9,
  "gasUsed": 10,
  "[INT_LT1M]": 11,
  "blockNumber": 12,
  "[INT_LT1B]": 13,
  "timestamp": 14,
  "2020-12-19T01:09:57Z": 15,
  "value": 16,
  "0.0": 17,
  "[INsSTART]": 18,
  "inputName": 19,
  "owner": 20,
  "type": 21,
  "address": 22,
  "inputValue": 23,
  "amount": 24,
  "uint256": 25,
  "87689390943463": 26,
```

(a) Output (transaction) vocabulary

```
"tree_vocab": {
  "[PAD]": 0,
  "[UNK]": 1,
  "0": 2,
  "0_1": 3,
  "0_1_1": 4,
  "0_1_1_1": 5,
  "0_1_2": 6,
  "0_1_1_2": 7,
  "0_1_1_3": 8,
  "0_1_1_4": 9,
  "0_1_1_5": 10,
  "0_1_1_6": 11,
  "0_1_1_7": 12,
  "0_1_1_8": 13,
  "0_1_1_9": 14,
  "0_1_1_10": 15,
  "0_1_1_11": 16,
  "0_1_1_12": 17,
  "0_1_1_13": 18,
  "0_1_1_13_1": 19,
  "0_1_1_13_2": 20,
  "0_1_1_13_3": 21,
  "0_1_1_13_4": 22,
  "0_1_1_13_5": 23,
  "0_1_1_13_6": 24,
```

(b) Tree vocabulary

```
"ctx_vocab": {
  "[PAD]": 0,
  "[UNK]": 1,
  "NONE": 2,
  "TO": 3,
  "FROM": 4
}
```

(c) Context vocabulary

7.2 Embedding

An embedding layer is a neural network layer that transforms categorical data (like words) into dense, continuous vector spaces where similar items are closer together*[Yad24]*.
An embedding layer has been applied to each tokenizer define previously.
To apply the embedding operation, some parameters have been specified, such as the dimension of the vector for each embedded token (m) and the number of tokens to embed (the size of the vocabulary). The result is an embedding with dimension equals to $n\ x\ m$.
Finally, the three different embeddings have been combined together through a sum operation, in order to allow the model to underline the characteristics of tokens, tree, and context of a given transaction.

7.3 GPT Architecture

GPT is built on Transformer networks, which are designed to process sequences (for example, sentences or longer passages) by modeling how each element relates to the others. Instead of reading input strictly step-by-step like older recurrent models, a Transformer uses attention to connect distant parts of the sequence efficiently.

The **input** is first split into small units called tokens (often words or subwords). Each token is mapped to a numeric vector through an embedding layer, so the model can operate in a continuous space rather than on raw text symbols. Positional information is then added (via positional embeddings) to preserve the order of the sequence, since attention alone does not inherently encode position.

Self-attention lets the model compute, for every position, which other tokens in the context are most informative, and to combine them into a context-aware representation. This makes it possible to capture relationships such as "which earlier phrase a later phrase refers to," even when they are far apart in the sequence. In GPT specifically, attention is causal: each token can only attend to tokens at earlier or equal positions, never to future ones.

GPT is not a single attention module but a **stack of many identical Transformer blocks**. Each block refines the representations produced by the previous one, progressively building higher-level features: early layers tend to capture simpler patterns, while deeper layers can represent more abstract dependencies. Multi-head attention is used inside each block so different heads can focus on different types of relationships simultaneously.

After the attention sublayer, each position is further transformed by a small **feed-forward network** (typically two linear layers with a nonlinearity). This component increases expressive power by recombining the features extracted by attention, position by position. Residual connections and layer normalization are used around these sublayers to stabilize training in deep networks.

At the end, GPT projects the final hidden representation into a probability distribution over the vocabulary and **predicts the next token**. Generation then proceeds iteratively: the model appends the predicted token to the context and repeats the same process to produce a continuation.

8. Implementation

This chapter details the actual implementation of the abnormality detection system. It is based on the architecture detailed in Chapter 7. The codebase developed is available at the repository *[Mar+26]*.

8.1 Technologies and Frameworks used

The abnormality detection system was entirely developed using Python. Python has established itself as the industry standard language for machine learning, artificial intelligence and data science due to its clean syntax, rapid prototyping capabilities and massive ecosystem. For a project that requires processing vast amounts of highly nested blockchain data, executing complex tokenization logic and training a custom Transformer model, Python offers an ideal balance of data manipulation flexibility and computational power.
Beyond its built-in capabilities for handling complex data structures, it also provides access to some powerful frameworks and libraries that form the backbone of our system, such as:

PyTorch The custom GPT-like Transformer architecture was built and trained using PyTorch. It was selected for its flexibility in defining multi-head attention layers and handling complex sequential data.

AMP To maximize computational throughput during the training phase, *Automatic Mixed Precision* (AMP) was enabled using PyTorch's *autocast* module. This optimization allows the model to significantly reduce memory usage and accelerate processing times without negatively affecting the model's accuracy.

Pandas The handling, structuring and manipulation of the massive volume of smart contract execution logs were powered by the Pandas library. Given the heterogeneity of the raw data, Pandas provided an efficient, DataFrame-oriented approach to align the diverse records into an unified schema. It facilitated crucial preprocessing operations before the data was forwarded to the tokenization module.

8.2 Repository Structure

The codebase separates data preprocessing, model definition, and execution into distinct components of the system, contained in different files within the *src* folder:

- **`tokenizer.py`:** This component converts raw transaction fields into discrete string tokens. It uses Pandas, regular expressions and Python's `ast` module to parse nested fields and literal evaluations. It implements a sophisticated normalization function that maps Ethereum addresses, transaction hashes and hexadecimal strings into standard tokens (e.g. `[ADDR]`). It also bucketized small integers to reduce vocabulary size, while intentionally preserving the exact values of medium integers to avoid losing anomaly reports based on specific amounts.
- **`dataset.py`:** This module defines PyTorch Dataset classes to handle sequence windowing. It implements `TxRandomWindowDataset` for the training phase, which randomly samples fixed-size token windows to encourage model generalization. For the inference phase, it provides `TxStrideWindowDataset`, which extracts windows sequentially using a predefined stride.
- **`engine.py`:** It contains the training and evaluation loops. It handles the forward passes using `autocast` and orchestrates backpropagation. It also provides the `score_transactions` function, which iterates a sliding window over a transaction sequence to compute the anomaly score based on per-tokens Negative Log-Likelihood (further discussed in Section 8.5).
- **`model.py`:** It contains the PyTorch class definitions. It defines `TripleEmbedding`, which combines standard output, hierarchical tree structures and contextual sender/receiver data by summing their embeddings. The `GPTModel` class utilized a `TransformerBlock` featuring multi-head attention, layer normalization and feed-forward networks.

8.3 Window-Based Architecture and Optimization

A core innovation in this implementation is the shift from token-by-token forward passes to a context-window approach. This strategy significantly reduces computational overhead and improves training stability.

8.3.1 Context Windows and Block Size

The model processes data in fixed-size windows defined by the `block_size` parameter, set to 128. This means the model considers a context of 128 tokens simultaneously to predict the subsequent token.

8.3.2 Training vs. Inference Strategy

The handling of these windows differs dynamically between training and evaluation phases to maximize performance:

- **Training Phase (Random Sampling):** During training, context windows are selected randomly along the transaction sequence. This acts as a form of data augmentation, forcing the model to generalize better by learning from various subsequences rather than memorizing a fixed order.
- **Inference Phase (Sequential Stride):** During evaluation, windows are extracted sequentially with a stride equal to the block size ($stride = 128$). For example, the first inference step covers tokens $[0-128]$, the second covers $[128-256]$,

and so on. This ensures that every token is evaluated exactly once without overlapping, minimizing redundant computations.

8.3.3 Computational Optimization

By aggregating 128 tokens into a single input tensor, the system parallelizes the prediction process. Instead of executing 128 separate forward passes and backpropagations, the model computes the loss for the entire window in a single pass. Furthermore, **Mixed Precision Training** (via `autocast`) has been enabled. This technique uses 16-bit floating-point numbers where possible to reduce memory usage and accelerate arithmetic operations without compromising model convergence.

8.4 Key Parameters and Their Influence

The system's performance relies heavily on the tuning of specific hyperparameters. The rationale behind the chosen values is described below:

Parameter	Value	Influence on Model Behavior
`Block Size`	128	Defines the "memory" length of the model. A size of 128 captures sufficient local context within a transaction to detect anomalies without incurring excessive memory costs associated with longer sequences.
`Batch Size`	16	Increased to 16 to stabilize gradient descent estimates. Larger batches improve training throughput on the GPU but require more VRAM.
`Stride`	128	Used during inference. Setting stride equal to block size ensures non-overlapping evaluation, making the anomaly score calculation deterministic and fast.

Table 8.1: Configuration of Key Hyperparameters

Additionally, the system is configured to save model checkpoints after every batch. While input-intensive, this ensures maximum resilience, allowing training to resume immediately from the latest state in case of interruption.

8.5 Anomaly Scoring

The detection of anomalies is based on the **Negative Log-Likelihood (NLL)**.

During the evaluation phase, the loss is calculated for every window in the transaction. These window-level losses represent the average per-token error within that block. To derive a final anomaly score for a full transaction, the system aggregates these values by computing the mean of all window losses.

$$Score_{transaction} = \frac{1}{N_{windows}} \sum_{i=1}^{N_{windows}} \mathcal{L}(w_i) \tag{8.1}$$

Where $\mathcal{L}(w_i)$ is the NLL of the i-th window. A higher score indicates that the transaction sequence has a low probability of occurring under the learned distribution, thus flagging it as an anomaly.

8.6 System Usage Instructions

To ensure reproducibility and ease of use, the system is designed to be executed via a single entry point. The following steps outline the practical usage of the framework.

Before initiating the training or validation pipelines, the execution datasets must be correctly positioned within the project directory:

- **Normal execution logs:** standard training and validation traces must be placed in the `data/` directory.
- **Attack logs:** the malicious transaction traces used exclusively for validation must be placed inside the `data/AttacksLogs/` folder.

The entire pipeline is then orchestrated by the `main.py` script. By default, running the following script executed both the training and validation phases.

```
python main.py
```

1. **Training Phase (`run_train=True`):**
   - The system first checks for existing tokenization caches in the `cache/` directory. If none are found, it builds the token sequences and vocabulary files (`vocabs.json`), saving them to disk to speed up future runs.
   - The model trains for the configured number of epochs (default is 10).
   - After each epoch, a state dictionary checkpoint is saved to the `models/` directory.
2. **Validation Phase (`run_validate=True`):**
   - The system loads the best-performing model checkpoint.
   - It computes the anomaly scores for the normal validation set and dynamically establishes the anomaly threshold at the 99th percentile.
   - It then ingests the attack logs, filtering out sequences that are too short to be evaluated.
   - Finally, it scores the attack transactions against the threshold and prints a comprehensive balanced evaluation to the standard output, detailing True Positives, False Positives, Precision, Recall, F1-Score, and a ranked list of the most severe attacks.

To experiment with different architectural dimensions or training dynamics, it is possible to directly modify the constants at the top of the `train_main()` and the `validation_main()` functions in `main.py`. Key parameters available for immediate tuning include `EMBEDDING_DIM`, `BLOCK_SIZE`, `BATCH_SIZE` and `EPOCHS`. Note that any changes to the tokenization logic require manually deleting the `cache/` directory to force a fresh preprocessing run.

9. Training and evaluation

Building on the architecture and implementation described in the previous chapters, the following sections presents the experimental methodology adopted to train and evaluate the proposed anomaly detection system. We describe the dataset composition and the train/validation/test split used to prepare the data for the model. We then analyze the impact of two distinct tokenization configurations, differing in the treatment of integer and address values, on convergence speed, throughput, and model quality. Finally, we evaluate the trained model against a curated set of real-world DeFi attack transactions, measuring its ability to distinguish malicious behavior from normal activity through a threshold-based anomaly scoring mechanism grounded in per-token Negative Log-Likelihood (NLL).

9.1 Setup

The model was trained on a dataset of **28,827 blockchain transactions**, obtained by merging two collections of normal blockchain activity logs, comprising 13,322 and 15,505 transactions respectively. The dataset was split with a fixed random seed (seed=42) into training (60%), validation (20%), and test (20%) sets. Each transaction was tokenized into a sequence of tokens and processed using a sliding window approach with a context length of 128 tokens. During training, windows were sampled randomly from the training sequences, yielding 32,000 samples per epoch organized into 2,000 batches of size 16. The validation set consisted of 29,751 fixed-stride windows used for consistent per-epoch evaluation.

Each training step processes a window of 128 tokens in a single forward pass and a single backpropagation, computing the cross-entropy loss as the mean of the per-token losses across the window. This aggregation strategy significantly reduces the number of optimization steps required per epoch while maintaining a fine-grained training signal at the token level. To further accelerate training, **automatic mixed precision** (AMP) was enabled via PyTorch's `autocast`, which performs computations in 16-bit floating point where possible, reducing memory usage and increasing throughput without affecting model accuracy. A model checkpoint was saved at the end of each epoch, allowing the best-performing configuration to be selected for the final evaluation.

The model, a single-block GPT-like Transformer with embedding dimension 64, 8 attention heads, and a feed-forward dimension of 256, was optimized using Adam with cross-entropy loss and mixed-precision training (AMP) on a CUDA-enabled GPU. Training ran for **10 epochs**. All experiments were conducted on the high-performance computing infrastructure provided by the University of Camerino, whose computational resources made it possible to run the training and evaluation pipelines at the scale required by this work.

Two experimental configurations were evaluated, differing in the tokenization strategy for integer values:

- **Config A — Encoded Integers**: both Ethereum addresses and integer values are replaced by special tokens (e.g. `[ADDR]`, `[INT_LT1M]`, `[BIGINT]`), yielding a compact vocabulary.
- **Config B — Clear Integers**: addresses are encoded, but integer values are kept as exact strings, producing a significantly larger vocabulary.

9.2 Training Results

The two configurations exhibit markedly different convergence behavior and throughput. Config A, thanks to its reduced vocabulary, processes tokens at approximately **7,024 tokens/s**, while Config B, burdened by the larger vocabulary of raw integer strings, reaches only **1,944 tokens/s**, resulting in a **3.6× slowdown**.

	Config A		**Config B**	
Epoch	**Train Loss**	**Val NLL/token**	**Train Loss**	**Val NLL/token**
1	1.9206	0.8213	2.6329	1.5986
2	0.5213	0.6115	1.0253	1.3410
3	0.4190	0.5545	0.8828	1.2559
4	0.3760	0.5216	0.8104	1.2068
5	0.3481	0.4987	0.7705	1.1811
6	0.3300	0.4843	0.7360	1.1584
7	0.3163	0.4690	0.7141	1.1465
8	0.3050	0.4580	0.6960	1.1285
9	0.2951	0.4509	0.6810	1.1150
10	0.2849	0.4419	0.6690	1.1020

Table 9.1: Per-epoch training loss and validation NLL/token for both configurations.

As shown in Table 9.1, Config A converges to a final validation NLL/token of **0.4419** after 10 epochs, compared to **1.1020** for Config B. This significant difference confirms that encoding high-cardinality integer values into bucketed tokens substantially improves the model's ability to learn the statistical structure of normal transactions, as the reduced vocabulary allows the model to generalize more effectively across rare numeric values.

9.3 Anomaly Detection Evaluation

The evaluation was performed using the final checkpoint of **Config A (epoch 9)** on a separate set of **24 real-world attack transactions** drawn from 19 known DeFi exploit files, including Beanstalk, Compound, MakerDAO, RariCapital, and others, scored against **5,493 normal validation transactions**. It is important to note that the `AttacksLogs` dataset was never exposed to the model during training: it serves exclusively as an external ground truth to assess the detection capabilities of the system on unseen malicious transactions.

The anomaly score assigned to each transaction is the **mean per-token NLL** `mean_token_nll`), computed by sliding a window of 128 tokens over the full transaction sequence, calculating the per-token NLL for each window, and averaging the resulting losses across all windows of the transaction. The detection threshold was set at the **99th percentile** of the validation score distribution, yielding a threshold of **1.6099**.

Metric	**Attacks** ($n = 24$)	**Normals** ($n = 5{,}493$)
Min	3.1673	0.3360
Median (p50)	6.0128	0.8194
p90	8.2989	1.3514
Max	8.5470	3.5217
Threshold	**1.6099**	

Table 9.2: Score distribution for attack and normal transactions.

The score distributions reported in Table 9.2 show a clear separation between attack and normal transactions. Even the lowest-scoring attack transaction (3.1673) lies well above the threshold, with a margin of +1.56 over the threshold itself. The resulting detection performance is reported in Table 9.3. Figure 9.1 further illustrates the top-10 highest-scoring attack transactions, including well-known DeFi exploits such as Beanstalk, Compound, MakerDAO, RariCapital, bZx, and Cryptopunk9998, all assigned anomaly scores well above the threshold of 1.6099.

Metric	**Value**
True Positives (TP)	24
False Positives (FP)	55
True Negatives (TN)	5,438
False Negatives (FN)	0
Recall	1.0000
Precision	0.3038
F1-Score	0.4660
Accuracy	0.9900

Table 9.3: Anomaly detection performance on real-world DeFi attack transactions.

The model achieves **perfect recall** (FN = 0), successfully flagging all 24 attack transactions. In total, 79 transactions were flagged as anomalous, of which 24 are true attacks and 55 are false positives. The false positive rate of 1.00% (55 out of 5,493 normal transactions) is a direct consequence of the threshold selection strategy: the threshold was set at the value above which 99% of the validation score distribution falls, meaning that by construction, approximately 1% of the validation set ($5{,}493 \times 0.01 \approx 55$ transactions) lies above it. In other words, the false positive rate directly reflects how the threshold was chosen based on the actual distribution of anomaly scores in the validation data. This trade-off is intentional in a security-critical context, where missing an attack (false negative) is far more costly than generating a false alarm. The low false negative rate, combined with the wide score margin between attacks and normal transactions, demonstrates that the model has effectively learned the statistical structure of legitimate blockchain activity and can reliably identify deviations consistent with known exploit patterns.

```
================= EVAL =================
Score metric: mean_token_nll
Threshold: 1.609871
N attacks: 24  N normals: 5493
TP=24  FP=55  TN=5438  FN=0
Precision=0.3038  Recall=1.0000  F1=0.4660  Acc=0.9900

Missed attacks (FN), lowest score first:

Top-10 attacks by score:
  global_attack_idx=2  score=6.780502  file=0x5efda50f22d34f262c29268506c5fa42cb56a1ce.json  local_tx=0
  global_attack_idx=23  score=6.230400  file=bZx_0x4f4e0f2cb72e718fc0433222768c57e823162152.json  local_tx=0
  global_attack_idx=10  score=6.126976  file=Compound_0xc0da02939e1441f497fd74f78ce7decb17b66529.json  local_tx=0
  global_attack_idx=11  score=6.126976  file=Compound_0xc0da02939e1441f497fd74f78ce7decb17b66529.json  local_tx=1
  global_attack_idx=12  score=6.126976  file=Compound_0xc0da02939e1441f497fd74f78ce7decb17b66529.json  local_tx=2
  global_attack_idx=17  score=6.033922  file=MakerDAO_0x55e8bef09624d60793181a125e07f040b4bcacbc.json  local_tx=0
  global_attack_idx=16  score=5.987325  file=Cryptopunk9998_0x9b5a5c5800c91af9c965b3bf06ad29caa6d00f9b.json  local_tx=0
  global_attack_idx=8  score=5.981434  file=Beanstalk_0x1c5dcdd006ea78a7e4783f9e6021c32935a10fb4.json  local_tx=0
  global_attack_idx=22  score=5.940287  file=RariCapital_0x67b66c99d3eb37fa76aa3ed1ff33e8e39f0b9c7a.json  local_tx=0
  global_attack_idx=15  score=5.615611  file=Compound_0xc0da02939e1441f497fd74f78ce7decb17b66529.json  local_tx=5
```

Figure 9.1: Top 10 detected attack transactions ranked by anomaly score

9.4 Overfitting Analysis

To assess whether the model generalizes well beyond the training data, we monitored the gap between training loss and validation NLL/token across all epochs. As reported in Table 9.1, both metrics decrease consistently throughout training, with the gap stabilizing at approximately 0.15 from epoch 3 onward without any subsequent divergence. This behavior indicates that the model does **not overfit**: the validation performance continues to improve in tandem with the training loss, suggesting that the learned representations generalize effectively to unseen transactions.

10. Conclusions and future work

This research has explored the feasibility and effectiveness of a runtime-oriented analytical framework for the detection of anomalous behaviors in smart contract executions through the structured analysis of EVM execution traces. By modeling smart contract interactions as sequences of behavioral signals rather than isolated transactions, the proposed approach demonstrates how richer semantic representations can significantly enhance the identification of subtle and complex anomalies that would otherwise remain undetected through traditional static or transaction-level analyses.

The experimental and conceptual findings suggest that a multi-layered representation of execution traces, enriched with contextual and structural information such as internal calls, gas consumption, input parameters, and emitted events, provides a robust foundation for anomaly detection in decentralized environments. In particular, the integration of a unified data representation module and attention-based layers enables the system to capture long-range dependencies and hidden behavioral correlations that are typical of modern DeFi protocols and composable smart contract ecosystems.

Another relevant contribution of this work lies in the alignment between training and inference phases. By enforcing a shared representation space for both historical and runtime data, the framework reduces the risk of distribution shift and ensures methodological coherence between offline learning and online monitoring. This design choice is especially important in blockchain environments, where execution dynamics evolve continuously and where static assumptions about contract behavior can quickly become obsolete.

Nevertheless, despite the promising results and the conceptual robustness of the proposed architecture, the rapidly evolving nature of decentralized finance, smart contract design patterns, and adversarial strategies highlights the necessity for further research and system-level extensions. The current framework should therefore be considered as a foundational step toward more adaptive, scalable, and proactive anomaly detection systems tailored to blockchain infrastructures.

Real-Time Detection and Proactive Mitigation

At the current stage, the system operates primarily on historical execution traces collected post hoc from blockchain data. While this approach is suitable for retrospective analysis and model training, it inherently limits the ability to intervene before an exploit is fully executed and permanently recorded on-chain. A natural and highly impactful extension of this research is the integration of the proposed model into a real-time monitoring architecture, such as a **Mempool Monitoring Service**.

By shifting the analysis from confirmed transactions to pending transactions residing in the mempool, the system could evaluate execution patterns before block inclusion.

This would enable near real-time anomaly scoring and the early identification of suspicious behavioral signatures, including flash-loan attacks, reentrancy patterns, or abnormal call-tree expansions. In such a scenario, the detection module could be connected to automated mitigation mechanisms, such as protocol-level circuit breakers, dynamic pausing functions, or alerting systems for developers and governance entities.

From a broader perspective, proactive mitigation introduces new research challenges related to latency constraints, computational efficiency, and false positive management. Real-time inference must be optimized to operate under strict time windows, while maintaining high reliability to avoid unnecessary contract interruptions. Therefore, future work should investigate lightweight embedding strategies, streaming inference pipelines, and adaptive thresholding techniques specifically designed for high-frequency blockchain environments.

Cross-Chain Anomaly Transfer

Another promising direction for future research concerns the generalization of anomaly detection models across multiple blockchain ecosystems. As the decentralized landscape progressively evolves toward a multi-chain paradigm, with the proliferation of Layer 2 solutions and EVM-compatible networks, the ability to transfer learned behavioral patterns across chains becomes increasingly relevant.

In this context, an interesting research objective is **Cross-Chain Generalization**, namely the evaluation of whether a model trained on Ethereum Mainnet execution traces can effectively detect anomalous behaviors on alternative infrastructures such as Arbitrum, Optimism, Polygon, or Avalanche. Since these environments share the underlying EVM execution logic, there exists a strong hypothesis that behavioral embeddings derived from execution traces may retain a significant degree of transferability.

However, several challenges must be addressed to validate this assumption. Different chains exhibit heterogeneous transaction volumes, gas dynamics, fee markets, and protocol compositions, all of which may influence execution trace distributions. Consequently, future studies should explore domain adaptation techniques, transfer learning strategies, and chain-specific normalization layers to preserve model performance while minimizing retraining costs. Such an extension would substantially increase the scalability and practical applicability of the proposed framework in real-world multi-chain monitoring systems.

Expansion of the Tree Layer Embedding

The current implementation of the Tree Layer focuses primarily on capturing the hierarchical structure of internal calls within the execution trace, enabling the model to understand the positional and relational context of nested contract interactions. While this representation already provides valuable structural insights, it can be further enriched to improve the granularity and interpretability of anomaly detection.

Future iterations of the Tree Layer could incorporate fine-grained gas-consumption metrics at each node of the call tree, including gas deltas, refund patterns, and opcode-level cost distributions. This enhancement would allow the model to detect more sophisticated behavioral anomalies, such as “Gas-Griefing” attacks, inefficient execution paths, or economically adversarial patterns that do not necessarily violate functional correctness but may still introduce systemic risks.

Moreover, the integration of temporal attributes and dynamic execution depth indicators could enable the model to distinguish between benign high-complexity executions and genuinely suspicious recursive or looping behaviors. This would be particularly relevant for DeFi protocols, where complex composability often generates deeply nested call structures that may superficially resemble anomalous patterns.

Additional Research Directions

Beyond the previously discussed extensions, several complementary research avenues could further strengthen the robustness and applicability of the proposed framework. First, the incorporation of explainability mechanisms, such as attention visualization and trace-level attribution scores, would improve the interpretability of anomaly decisions and facilitate their adoption in security auditing workflows.

Second, the integration of hybrid analysis techniques that combine runtime trace embeddings with static code features could provide a more holistic understanding of smart contract behavior. This dual perspective would allow the system to correlate latent behavioral anomalies with specific code segments, improving both detection accuracy and forensic analysis capabilities.

Finally, future work should also address scalability and dataset standardization challenges. As blockchain datasets continue to grow in size and heterogeneity, efficient data pipelines, unified trace schemas, and reproducible benchmarking protocols will become essential to ensure the long-term sustainability of research in this domain. In this sense, the development of open, normalized execution-trace datasets could significantly accelerate progress in AI-driven smart contract security.

Bibliography

[Boa22] Agenda Digitale Editorial Board. "DeFi, allarme cybersecurity: gli attacchi più comuni e le contromisure". In: *Agenda Digitale* (2022).

[Con+23] Lin William Cong et al. "Crypto wash trading". In: *Management Science* 69.11 (2023), pp. 6427–6454.

[Cor+25] Flavio Corradini et al. "A methodology for extracting and decoding smart contracts data, Computer Communications". In: 2025.

[Dev+19] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.

[DR+25] Juri Di Rocco et al. "On the use of large language models in model-driven engineering". In: *Software and Systems Modeling* (2025), pp. 1–26.

[Far22] Beanstalk Farms. *Beanstalk: A Permissionless Fiat Stablecoin Protocol*. Tech. rep. Beanstalk, 2022.

[Fou21] MakerDAO Foundation. *Authorization Model and the hope() Function*. Tech. rep. MakerDAO, 2021.

[Gai+23] Yu Gai et al. "Blockchain large language models". In: *arXiv preprint arXiv:2304.12749* (2023).

[Gro23] OSWAR Working Group. "Governance Exploit (DAO takeover) – Beanstalk Example". In: *Open Standard Web3 Attack Reference*. 2023.

[Hu+23] Sihao Hu et al. "Bert4eth: A pre-trained transformer for ethereum fraud detection". In: *Proceedings of the ACM Web Conference 2023*. 2023, pp. 2189–2197.

[Mad16] Vijay Madisetti. "Blockchain Platform for Industrial Internet of Things". In: *Journal of Software Engineering and Applications* (2016).

[MAD22] Bruno Mazorra, Victor Adan, and Vanesa Daza. "Do not rug on me: Leveraging machine learning techniques for automated scam detection". In: *Mathematics* 10.6 (2022), p. 949.

[Man+25] Felipe Alejandro Manzor Manzor et al. "Suspicious Activity Detection Using Blockchain Process Mining". In: *International Conference on Advanced Information Systems Engineering*. Springer. 2025, pp. 129–141.

[Mar+26] Diego Marinangeli et al. *AbnormalityDetection*. `https://github.com/ammermali/AbnormalityDetection`. 2026.

[Mes+23] Joao Messias et al. "Understanding Blockchain Governance: Analyzing Decentralized Voting to Amend DeFi Smart Contracts". In: *arXiv preprint arXiv:2305.17655* (2023).

[Nak08] Satoshi Nakamoto. "Bitcoin: A peer-to-peer electronic cash system". In: *Available at SSRN 3440802* (2008).

[Onu+23] Ifeyinwa Jacinta Onu et al. "Detection of Ponzi scheme on Ethereum using machine learning algorithms". In: *Scientific Reports* 13.1 (2023), p. 18403.

[Rac25] Yoesoep Edhie Rachmad. "CBDC: Integrating Blockchain Technology into the Traditional Financial System". In: *The United Nations and the Nobel Peace Prize Awards* (2025).

[Sta22] Bitcoin Insider Staff. "Beanstalk Farms loses $182M Through Governance Exploit". In: *Bitcoin Insider* (2022).

[Sun+24] Yuqiang Sun et al. "Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis". In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 2024, pp. 1–13.

[Suz+] Iori Suzuki et al. "DeFiIntel: A Dataset Bridging On-Chain and Off-Chain Data for DeFi Token Scam Investigation". In: ().

[Sza97] Nick Szabo. "Formalizing and securing relationships on public networks". In: *First monday* (1997).

[TS+19] Christof Ferreira Torres, Mathis Steichen, et al. "The art of the scam: Demystifying honeypots in ethereum smart contracts". In: *28th USENIX Security Symposium (USENIX Security 19)*. 2019, pp. 1591–1607.

[Vas+17] Ashish Vaswani et al. "Attention is all you need". In: *Advances in neural information processing systems* 30 (2017).

[Wan+22] Dong Wang et al. "Penny Wise and Pound Foolish: Quantifying the Risk of Unlimited Approval of ERC20 Tokens on Ethereum". In: *arXiv preprint arXiv:2207.01790* (2022).

[WLZ21] Qiang Wang, Rongrong Li, and Lina Zhan. "Blockchain technology in the energy sector: From basic research to real world applications". In: *Computer Science Review* 39 (2021), p. 100362.

[Yad24] Amit Yadav. "How Does Embedding Layer Work?" In: *Medium* (2024).

[Yu+25] Jiahao Yu et al. "BlockScan: Detecting Anomalies in Blockchain Transactions". In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025.

[Zhu+20] Yuan Zhuang et al. "Smart Contract Vulnerability Detection using Graph Neural Network". In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*. Ed. by Christian Bessiere. Main track. International Joint Conferences on Artificial Intelligence Organization, July 2020, pp. 3283–3290. DOI: 10.24963/ijcai.2020/454. URL: https://doi.org/10.24963/ijcai.2020/454.

Acknowledgments

This study would not have been possible without the invaluable guidance and support of our professors, Prof. Flavio Corradini, Prof. Alessandro Marcelletti and Dr. Filippo Lampa. Their tireless dedication and highly qualified insights provided us with the necessary academic foundation to explore the complexities of blockchain security, providing critical perspectives that significantly shaped the development of this research project. A special thanks goes to the University of Camerino for the excellent academic preparation provided and for the professional experiences enabled throughout our studies.